A Quarter of the Budget Is Spent Twice: Duplicated Evidence in Budgeted Context Packing and a Simple Fix

Anonymous ACL submission

Abstract

When a long document must be answered from under a hard token budget, the standard recipe ranks chunks with a retriever, pads each with neighbouring text, and concatenates the top ones until the budget runs out. Research has focused on the ranking step; we look at the packing step and find that it is surprisingly wasteful. On LongBench, standard dense chunk packing spends 23% of a 1,024-token budget, and 29% of a 2,048-token budget, on text that already appears elsewhere in the same context, because neighbouring top-ranked chunks are padded into each other; almost every packed piece also ends mid-sentence. We present DEDUPPACK, a training-free packer that removes this waste behind any ranker: it extends each selected span to sentence boundaries, merges overlapping extensions so that no text is paid for twice, admits evidence under the exact token count of the downstream tokenizer with backfilling, and emits it in document order. Duplication drops to 1.4%, and the recovered budget raises answer coverage by 3–4 points. We freeze every design choice on a development split and run the LongBench test set (1,150 single- and multi-document QA instances) once, with three instruction models (Qwen2.5-7B, Llama-3.1-8B, Qwen2.5-14B) and three budgets (512, 1,024, 2,048 tokens). DEDUPPACK outperforms dense BGE chunk packing in all nine model–budget settings, by 1.9–3.7 F1 when pooled over models ($p \leq 0.001$ at every budget), and outperforms a structured lexical pair packer by 2.5–3.1 F1. The gain does not shrink with better rankers: holding the ranking fixed, DEDUPPACK improves chunk packing by 2.8, 3.8 and 2.0 F1 with BGE-small, BGE-large and a cross-encoder reranker, and it outperforms LlamaIndex's sentence-window and auto-merging retrievers and LongLLMLingua. With one sixth of the input (2,048 tokens vs. 12.5K on average), DEDUPPACK retains 99–101% of full-context F1 on all three models (no significant difference), shrinking the per-request KV cache about sixfold (e.g., 2.4 GB→0.4 GB for Qwen2.5-14B); on InfiniteBench, 1,024 tokens—1/30 of the 30K-token full context—retain 100–104%. At the same budget it exceeds LLMLingua-2 by 15.3 F1.

1 Introduction

Large language models (LLMs) are routinely asked to answer questions about long documents and document collections. Even when the context window is long enough, there are reasons to send the model less text: prefill cost and latency grow with prompt length, many deployed models have short windows, and long inputs dilute the few passages that matter (Liu et al., 2024a). In practice a long context is therefore often reduced to a *hard* budget of a few hundred to a few thousand tokens, measured in the tokenizer of the model that will read it.

Two families of methods do this. *Prompt compressors* (Li et al., 2023; Jiang et al., 2023, 2024; Pan et al., 2024) delete tokens or phrases judged uninformative. *Retrieval-style packers* split the context into chunks or spans, rank them with BM25 or a dense encoder (Robertson and Zaragoza, 2009; Karpukhin et al., 2020; Xiao et al., 2023b), and concatenate the top-ranked units, each padded with a fixed window of neighbouring tokens, until the budget is exhausted. Research effort has concentrated on the ranking step; the packing step is treated as bookkeeping.

We start from a simple question: *where does the budget go?* Measuring the contexts produced by standard dense chunk packing on LongBench (Section 3), we find that relevant chunks cluster—the passage that answers a question tends to contain several top-ranked chunks—so their padded windows overlap, and the same sentences are rendered and paid for several times. At a 1,024-token budget, 23% of the packed text duplicates text elsewhere in the same context, and the share grows with the budget (15%, 23% and 29% at 512, 1,024 and 2,048

tokens). In addition, 94% of packed pieces end in the middle of a sentence, and greedy admission leaves part of the budget unused. Under a hard budget, every duplicated or broken token is a token not spent on evidence.

We then show that this waste can be removed without touching the ranker. DEDUPPACK is a packer that can be placed behind any span ranker (Figure 1). Its core operation is *deduplication by merging*: the padded ranges of all selected spans that touch or overlap are merged into a single passage, so every token of the source is paid for at most once, and the cost of a new span is the number of tokens it actually adds. Around this core, DEDUPPACK admits spans in rank order under the exact downstream token count and *backfills* (keeps admitting later spans after one does not fit), aligns the padding to sentence boundaries, and emits the merged passages verbatim in document order. None of these operations requires training, and all of them run on the CPU before the model is called.

We evaluate DEDUPPACK under a protocol designed to leave little room for doubt. Every design choice, including the ranker, was fixed on a development split built from LongBench-E with all LongBench test questions removed; the LongBench test set was then run once. All methods share the candidate spans, the prompt wrapper, the model, greedy decoding, the scorer, and a budget counted with the downstream tokenizer. We test three instruction models and three budgets and report paired bootstrap tests. Our findings are:

- **Standard packing wastes a quarter of the budget.** Dense chunk packing duplicates 15–30% of its context; DEDUPPACK reduces this to about 1.5% and raises answer coverage by 3–4 points at every budget (Section 3).

- **Removing the waste beats ranking-only improvements.** DEDUPPACK outperforms dense BGE chunk packing in all nine model–budget settings, by +3.68, +1.92 and +2.26 F1 at 512, 1,024 and 2,048 tokens (pooled over models, all $p \leq 0.001$), and outperforms a structured lexical pair packer by 2.5–3.1 F1 (Tables 2 and 3).

- **Better rankers do not remove the waste.** With BGE-large and with a cross-encoder reranker, swapping only the packer improves F1 by 3.8 and 2.0 ($p \leq 0.001$); DEDUPPACK with the reranker is the best compressed configuration and outperforms LlamaIndex's sentence-window and auto-merging retrievers, Provence and LongLLMLingua (Section 6.3). The direction carries over to book-length InfiniteBench contexts (Section 6.5).

- **The gain is retriever-agnostic.** Holding the BGE ranking fixed and changing only the packer improves F1 by 2.5–2.8 at every budget; with a purely lexical ranker DEDUPPACK still outperforms the pair packer that uses the same lexical signals (Section 6.2).

- **The gain extends beyond QA and is cheap.** Over all eleven query-bearing LongBench tasks, DEDUPPACK outperforms chunk packing by 5.1 F1 ($p < 0.001$); it runs on one CPU core in 0.4 s per instance, about 40× faster than LongLLMLingua (Sections 6.4 and 6.9).

- **One sixth of the tokens keeps full-context accuracy.** A 2,048-token DEDUPPACK context (vs. 12.5K on average) retains 99–101% of full-context F1 on all three models (paired differences $+0.56$, -0.35 and -0.10 F1, none significant), which cuts the per-request KV cache about sixfold; on InfiniteBench, 1,024 tokens retain 100–104% of a 30K-token full context (Section 6.5).

- **Evidence-level packing dominates token deletion.** At the same exact budget, DEDUPPACK exceeds LLMLingua-2 by 15.3 F1.

- **Deduplication is the active ingredient.** In a build-up ablation with a fixed ranker, merging alone recovers 77% of the gain (+2.11 F1, $p < 0.001$, significant at every budget and on each model), removing it from the full packer costs 1.71 F1 ($p < 0.001$), and the remaining operations add small, non-significant amounts. Instances with more duplication benefit more (Section 6.8).

2 Related Work

Prompt and context compression. Selective Context removes lexical units with low self-information (Li et al., 2023). LLMLingua and LongLLMLingua estimate token importance with a small LM, the latter with question-aware coarse-to-fine compression (Jiang et al., 2023, 2024);

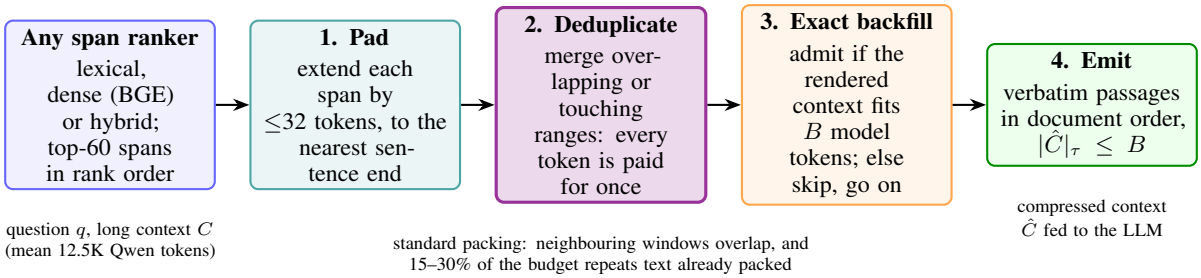


Figure 1: DEDUPPACK packs the spans returned by any ranker. Each span is padded up to a sentence boundary, overlapping padded ranges are merged so that no text is paid for twice, spans are admitted in rank order under the exact downstream token budget with backfilling, and the result is emitted verbatim in document order. DEDUPPACK runs before prefill and does not modify the model.

LLMLingua-2 distils keep/drop labels into a bidirectional encoder (Pan et al., 2024). CPC scores sentences against the question with a trained encoder (Liskavets et al., 2025), and RECOMP and CompAct train extractive or abstractive compressors (Xu et al., 2024; Yoon et al., 2024). DEDUPPACK is training-free, never edits text inside a kept passage, and is complementary to learned scorers: any of them can supply its ranking.

Retrieval-style context construction. Retrieval-augmented generation ranks passages with sparse (Robertson and Zaragoza, 2009) or dense (Karpukhin et al., 2020; Xiao et al., 2023b) retrievers and concatenates the top results (Lewis et al., 2020; Izacard and Grave, 2021); diversity-aware selection (Carbonell and Goldstein, 1998) and multi-hop retrieval (Xiong et al., 2021; Trivedi et al., 2023) refine *which* units are selected. We study the complementary question of how selected units are turned into a context under an exact budget, and show that this step alone moves F1 by 2–4 points. Practitioners have observed that overlapping chunks can return near-duplicate neighbours that waste prompt budget, and an industrial system limits the index gap between selected chunks (Sundar and Morabia, 2026); exact deduplication of identical chunks or documents has been measured in RAG corpora (Schelpe, 2026), and sentence-window retrieval has been reported to underperform fixed-size chunks (Eibich et al., 2024). To our knowledge, the share of the budget that standard packing spends on duplicated text within a single long document, and its effect on accuracy, have not been quantified before.

Long-context evaluation and KV-cache compression. LongBench (Bai et al., 2023) evaluates QA over long inputs, and models under-use evidence in the middle of long contexts (Liu et al., 2024a). KV-cache methods evict, merge or quantise cached states during generation (Xiao et al., 2023a; Zhang et al., 2023; Li et al., 2024; Liu et al., 2024b). DEDUPPACK instead shortens the input before prefill, so the full context is never encoded.

B	Packer	Dup. ↓	Sent. end ↑	Tokens	Ans. cov. ↑
512	BGE chunks	14.7	6.7	473	30.4
	Pair packing	3.0	6.2	439	30.0
	DEDUPPACK	**1.1**	**70.6**	500	**33.9**
1,024	BGE chunks	22.8	6.3	985	39.0
	Pair packing	5.4	6.0	920	36.9
	DEDUPPACK	**1.4**	**68.1**	1,014	**41.9**
2,048	BGE chunks	29.1	6.2	2,010	45.0
	Pair packing	7.3	6.1	1,502	41.2
	DEDUPPACK	**1.6**	**62.8**	2,032	**49.0**

Table 1: Where the budget goes (LongBench test, 1,150 instances, Qwen tokenizer; Llama-3.1 is within one point on every column). Dup.: share of packed words that start an 8-word sequence already present earlier in the same context. Sent. end: share of packed pieces ending with sentence punctuation. Tokens: mean context length. Ans. cov.: share of instances whose normalised gold answer appears verbatim in the context.

3 Where Does the Budget Go?

We measure three kinds of waste in the contexts produced by standard packers on the LongBench test set (Section 5), before any model reads them (Table 1).

Duplication. We count a packed word as duplicated if the 8-word sequence it starts already appeared earlier in the same packed context. Dense chunk packing duplicates 14.7%, 22.8% and 29.1% of its text at 512, 1,024 and 2,048 tokens. The cause is structural: the top-ranked candidate spans for a question are rarely spread evenly over the document; they cluster in the passages that are ac-

tually relevant, and adjacent spans overlap by construction (64-token spans with stride 32). Padding each span by ±32 tokens then makes neighbouring windows overlap almost completely. The larger the budget, the more neighbours are admitted, and the larger the duplicated share. A structured pair packer that penalises redundancy at selection time reduces duplication but does not remove it (3–7%), because overlap is created by the padding, not by the selection.

Broken sentences. Only about 6% of the pieces produced by fixed-window packing end with sentence punctuation; the remaining 94% stop mid-sentence, often inside the phrase that answers the question.

Unused budget. Greedy admission that stops or skips on long units leaves part of the budget unfilled: on average 39 of 1,024 tokens for chunk packing and 104 for pair packing.

Consequence. The three effects combine into lower answer coverage: at every budget, the gold answer is less often present in the packed context than it could be. The packer we describe next removes all three: duplication drops to 1.1–1.6%, two thirds of pieces end on a sentence boundary, the budget is filled to within 10 tokens, and answer coverage rises by 2.9–4.0 points over dense chunk packing with the same budget.

4 Method

4.1 Problem Setting

Given a question q and a long context C, we construct $\hat{C}$ with $|\hat{C}|_\tau \leq B$, where $|\cdot|_\tau$ counts tokens with the tokenizer τ of the downstream model. A fixed prompt wrapper keeps the task instruction and question unchanged; only the context slot is replaced by $\hat{C}$. Because the budget is defined in downstream tokens, every method is held to exactly the same constraint for each model.

4.2 Candidate Spans

The context is split into contiguous pages of 256 regex tokens (words or single punctuation marks), and each page into candidate spans of 64 regex tokens with stride 32; spans never cross pages. Regex tokens only define candidate units; all budget accounting uses τ. All span-based methods in our comparison use the same candidate spans.

4.3 Rankers

DEDUPPACK accepts any function that scores spans against q. We use three, none of which sees gold answers. The *lexical* ranker fuses z-normalised BM25 (k_1=1.2, b=0.75) and TF-IDF scores computed over the spans of the instance. The *dense* ranker is the cosine similarity between the question (with BGE's query instruction) and the span under `bge-small-en-v1.5` (Xiao et al., 2023b). The *hybrid* ranker adds the z-normalised lexical and dense scores. The top 60 spans form the priority list P, ordered by score with ties broken by position. Our main configuration uses the hybrid ranker, chosen on the development split; Section 6.2 shows the gains do not depend on this choice.

4.4 The Packer

Deduplication by merging (core). Top-ranked spans cluster in the relevant passages, and their padded ranges overlap (Section 3). DEDUPPACK keeps the selection as a set of token ranges, sorts them by page and position, and merges every pair that overlaps or touches into one range. Overlapping evidence is therefore rendered, and paid for, once, and a span adjacent to already selected evidence costs only the tokens it adds.

Sentence-aligned padding. As in standard packing, each span is padded with context, by at most $r = 32$ regex tokens per side and within its page. Instead of a fixed window, DEDUPPACK stops the padding at the nearest sentence terminator (`.`, `?`, `!`) within that radius, so padded ranges are never longer than the fixed window and usually end on a sentence boundary. This operation mainly improves readability; Section 6.8 shows that its effect on accuracy is small.

Exact admission with backfill. Spans are considered in priority order. A span is admitted if the context rendered from the current selection plus the span (padded, merged and joined after the prefix "Question: q / Selected evidence:") has at most B tokens under τ. Because merging can make a new span nearly free, the cost of a span depends on what is already selected; admission therefore measures the rendered, deduplicated context rather than summing per-span lengths. If a span does not fit, DEDUPPACK skips it and continues with the next one, so short spans further down the list can still use the remaining budget.

Document-order emission. The merged passages are emitted verbatim in document order, separated by blank lines. If re-joining changes the token count, the context is clipped to B, so $|\hat{C}|_\tau \leq B$ always holds. Algorithm 1 (Appendix A) gives the procedure.

Development protocol. All choices—the ranker, sentence rather than fixed repair, r, the list length, merging, backfilling and emission order—were made on a development split of 584 instances: the HotpotQA and 2WikiMultihopQA portions of LongBench-E (Bai et al., 2023) after removing every question that also appears in the LongBench test set. The development experiments used a 4-bit Qwen2.5-7B model on a separate GPU. The configuration was then frozen and the test set was run once with every model and budget reported here; no test result was used to change the method.

5 Experimental Setup

Data. All English single- and multi-document QA tasks of LongBench (Bai et al., 2023): Qasper (Dasigi et al., 2021) (200), MultiFieldQA-en (150), HotpotQA (Yang et al., 2018) (200), 2WikiMultihopQA (Ho et al., 2020) (200), MuSiQue (Trivedi et al., 2022) (200) and NarrativeQA (Kočiský et al., 2018) (200), 1,150 instances. We report LongBench's F1 and the macro average over the six tasks; "MH" averages the three multi-hop tasks.

Models and decoding. Qwen2.5-7B-Instruct, Qwen2.5-14B-Instruct (Qwen Team, 2024) and Llama-3.1-8B-Instruct (Grattafiori et al., 2024), in bfloat16 with vLLM, LongBench's task prompts inside each model's chat template, greedy decoding and LongBench's per-task output lengths. The 14B model reads the packed contexts built with the Qwen tokenizer, which it shares with the 7B model.

Budgets. $B \in \{512, 1{,}024, 2{,}048\}$ downstream tokens for the context slot. The mean full prompt has 12.5K Qwen tokens, so these budgets keep roughly 4%, 8% and 16% of the context.

Baselines. All baselines fill the same slot under the same exact budget and tokenizer. *BGE chunks*: the 64 spans most similar to the question under BGE, each padded by ± 32 tokens, admitted in similarity order while they fit and emitted in that order; this is the standard dense packing recipe. *Pair packing*: a structured lexical packer that selects query–support span pairs with the lexical ranker, a lexical bridge score and a redundancy penalty, repairs each span with a fixed ± 32 window, admits pairs until the first one that does not fit and emits them in document order. It represents structured, multi-hop-aware selection with conventional packing. *LLMLingua-2* (Pan et al., 2024) (`xlm-roberta-large-meetingbank`), asked to compress the context to the budget; its output is placed after the same prefix and truncated to B downstream tokens if needed. *Full context*: the whole context, middle-truncated to 30K tokens as in LongBench (a 32K window for all models), as a reference. Section 6.3 adds stronger rankers (BGE-large; `bge-reranker-v2-m3` over the top 100 BGE spans) and published systems: LlamaIndex's sentence-window and auto-merging retrievers (Liu, 2022) (official node parsers, default window 3 and hierarchy 2048/512/128, merge threshold 0.5, BGE-small retrieval), sentence-aligned chunks of up to 64 words ranked by BGE, Provence (Chirkova et al., 2025) applied to the top 60 BGE chunks, and LongLLMLingua (Jiang et al., 2024) with its recommended question-aware settings. For LongLLMLingua we use Llama-3.1-8B-Instruct as the compressor LM; it failed on 26 of 1,150 instances (2.3%), for which we fall back to the head of the context. All of these are packed into the same exact budget; units are admitted in their own rank order while they fit. Appendix E compares the pair packer with truncation, Selective Context and BM25/TF-IDF chunk packing in an earlier environment.

Statistics. Comparisons are paired on instances. We use a task-stratified paired bootstrap (Efron and Tibshirani, 1993) with 10,000 resamples and seed 20260701; the statistic is the macro average of per-task mean differences. Pooled results resample within the eighteen model–task strata. We report 95% percentile intervals and two-sided p-values.

6 Results

6.1 DedupPack versus standard packing

Table 2 gives macro F1 for every model and budget, and Table 3 the pooled paired tests. DedupPack outperforms BGE chunk packing in all nine model–budget settings. Pooled over models, the gain is +3.68 F1 at 512 tokens, +1.92 at 1,024 and +2.26 at 2,048, and every interval excludes zero. Per model, eight of the nine differences are significant at $p < 0.05$ (the ninth, Qwen2.5-7B at 1,024 tokens, is +1.79 with $p = 0.06$; Appendix D). The gain is

Method	Qwen2.5-7B			Llama-3.1-8B			Qwen2.5-14B			Avg.
	512	1,024	2,048	512	1,024	2,048	512	1,024	2,048	
Full context (reference)		43.04			43.13			47.59		
LLMLingua-2	–	24.60	–	–	28.43	–	–	27.10	–	–
Pair packing	33.88	39.11	40.68	34.05	37.79	39.60	36.73	41.69	44.23	38.64
BGE chunks	33.05	38.85	40.98	33.27	38.99	40.85	36.27	42.34	45.29	38.88
DEDUPPACK (BGE ranker)	35.64	**40.76**	43.14	36.16	**41.67**	**43.58**	39.21	**45.18**	**48.10**	**41.49**
DEDUPPACK (hybrid ranker, main)	**36.74**	40.64	**43.60**	**36.44**	40.98	42.78	**40.44**	44.32	47.50	**41.49**

Table 2: Macro F1 on the 1,150 LongBench QA instances under exact budgets of 512, 1,024 and 2,048 downstream tokens. The full-context reference reads up to 30K tokens. Bold marks the best compressed result in each column. Per-task scores are in Appendix C.

DEDUPPACK minus	512	1,024	2,048
BGE chunks	**+3.68** [2.53, 4.82]	**+1.92** [0.87, 2.97]	**+2.26** [1.24, 3.29]
Pair packing	**+2.99** [2.04, 3.94]	**+2.45** [1.52, 3.39]	**+3.12** [2.18, 4.08]
Same BGE ranking, different packer:			
DEDUPPACK-BGE − BGE chunks	**+2.81** [1.88, 3.76]	**+2.48** [1.50, 3.45]	**+2.57** [1.58, 3.57]

Table 3: Paired F1 differences pooled over the three models (95% bootstrap intervals). All intervals exclude zero ($p \leq 0.001$). Per-model tests are in Appendix D.

Ranker	Packing / system	Qwen-14B	Llama-8B	Δ (pooled)
BGE-small	chunks	42.34	38.99	
	DEDUPPACK	45.18	41.67	**+2.76*****
BGE-large	chunks	42.55	40.12	
	DEDUPPACK	46.23	43.98	**+3.77*****
BGE + bge-reranker-v2-m3	chunks	44.37	42.21	
	DEDUPPACK	**46.45**	**44.12**	**+2.00*****
LlamaIndex sentence window		39.99	36.96	
LlamaIndex auto-merging		41.50	39.13	
Sentence chunks + BGE		44.24	39.98	
Provence (BGE + pruning)		44.27	40.69	
LongLLMLingua		40.66	39.67	
LLMLingua-2		27.10	28.43	
Full context (30K)		47.59	43.13	

Table 4: Stronger rankers and external context-construction systems (LongBench, 1,024 tokens, macro F1). Δ: DEDUPPACK minus chunk packing *with the same ranking*, pooled over the two models (***: $p \leq 0.001$; significant for each model separately as well).

largest at the smallest budget, where every wasted token is most costly.

DEDUPPACK also outperforms the structured pair packer by 2.5–3.1 F1 at every budget ($p < 0.001$), in all nine settings. The pair packer was designed for multi-hop questions and selects evidence in pairs, yet a packer that ranks spans one at a time but packs them carefully is better on the multi-hop tasks as well: at 1,024 tokens DEDUPPACK reaches 46.3, 43.9 and 50.3 MH F1 on the three models, against 42.4, 39.6 and 47.0 for pair packing and 42.7, 41.3 and 48.0 for BGE chunks (Appendix C).

6.2 The packer, not the ranker, carries the gain

The most direct test of the packer is to keep the ranking fixed. BGE chunk packing and DEDUPPACK with the BGE ranker use the same BGE similarities over the same spans and differ only in how the top spans are turned into a context. Changing the packer improves F1 by +2.81, +2.48 and +2.57 at the three budgets (Table 3), and per model it is significant in all nine settings (Appendix D).

Conversely, the choice of ranker inside DEDUPPACK matters little. The hybrid ranker selected on the development split and the BGE ranker are within 0.6 F1 of each other on average and their difference is not significant (-0.56, $[-1.49, 0.40]$ at 1,024 tokens); each is best in some settings. With a purely lexical ranker, DEDUPPACK outperforms the pair packer, which uses the same lexical signals, by 1.54 F1 [0.68, 2.43] ($p < 0.001$) at 1,024 tokens. In short, the improvement comes from the packing operations and transfers across lexical, dense and hybrid rankers.

6.3 Stronger rankers and external systems

A natural question is whether the waste we identify matters only for weak rankers. Table 4 repeats the packer swap with two stronger rankers on Qwen2.5-14B and Llama-3.1-8B. With BGE-large, DEDUPPACK improves on chunk packing by 3.77 F1 [2.67, 4.88]; with a cross-encoder reranker (`bge-reranker-v2-m3` over the top 100 BGE spans) by 2.00 [0.85, 3.12]. Both gains are significant on each model separately. A better ranker therefore does not remove the packing waste: the gain from better packing is at least as large as

Method (1,024 tokens)	5 new tasks		All 11 tasks	
	Qwen-14B	Llama-8B	Qwen-14B	Llama-8B
Pair packing	58.39	42.94	49.28	40.13
BGE chunks	54.64	39.98	47.93	39.44
DEDUPPACK (BGE)	61.81	47.20	**52.74**	44.18
DEDUPPACK (hybrid, frozen)	**62.56**	**49.67**	52.61	**44.93**
Full context (30K)	63.10	46.59	54.64	44.70

Table 5: All query-bearing English LongBench tasks: the six QA tasks plus query-based summarisation (QMSum), few-shot learning (TriviaQA, TREC, SAMSum) and passage retrieval (PassageRetrieval-en), 200 instances each. Per-task scores are in Appendix C.

Method	Qwen2.5-14B		Llama-3.1-8B	
	1,024	2,048	1,024	2,048
Pair packing	33.71	34.61	32.80	36.55
BGE chunks	36.58	37.44	36.81	40.15
Sentence chunks + BGE	36.36	37.69	36.09	39.99
DEDUPPACK (hybrid, frozen)	**38.03**	**38.45**	**38.36**	40.98
DEDUPPACK (BGE)	36.76	37.63	36.56	**41.49**
Full context (30K)	36.44		38.16	

Table 6: InfiniteBench En.QA and En.MC (Zhang et al., 2024) (580 questions over novels of 150K+ tokens), mean of QA F1 and MC accuracy. No setting was tuned on this benchmark.

the gain from replacing BGE-small with BGE-large or with a reranker, and the two add up. DEDUPPACK with the reranker is the best compressed configuration on both models, and on Llama-3.1-8B it scores above the 30K-token full context (44.12 vs. 43.13).

We also compare with published context-construction systems under the same budget and wrapper. LlamaIndex's sentence-window retrieval (Liu, 2022), which retrieves single sentences and replaces each with its ±3-sentence window, trails the frozen DEDUPPACK configuration by 4.18 F1 [2.76, 5.65]; its windows of neighbouring top sentences overlap heavily, which is exactly the duplication of Section 3. LlamaIndex's auto-merging retriever, which replaces retrieved leaf chunks by their parent when most of the parent's children are retrieved, trails it by 2.33 [0.94, 3.79]: merging at fixed hierarchy levels removes some duplication but admits whole parents that are mostly irrelevant. LongLLMLingua (Jiang et al., 2024) trails it by 2.48 [1.02, 3.98]. Provence (Chirkova et al., 2025), a trained pruner that removes irrelevant sentences from each retrieved chunk, is close to the frozen configuration (+0.17, n.s.) and is clearly below DEDUPPACK given a reranker (+2.80 [1.49, 4.16]); DEDUPPACK needs no trained pruner. Finally, packing non-overlapping sentence-aligned chunks with BGE, which avoids duplication by construction, recovers part of the gain, as our diagnosis predicts; DEDUPPACK with the same BGE ranker remains 1.31 F1 higher [−0.03, 2.67].

6.4 Beyond question answering

LongBench contains five further English tasks that come with a query: query-based meeting summarisation (QMSum), three few-shot tasks whose context is a long list of demonstrations (TriviaQA, TREC, SAMSum), and passage retrieval, which asks which of 30 paragraphs a given summary describes. We run them with the frozen configuration and no change (Table 5). Over all eleven query-bearing tasks, DEDUPPACK outperforms chunk packing by 5.08 F1 [4.18, 6.00] and the pair packer by 4.06 [3.21, 4.89] pooled over the two models (both $p < 0.001$, and on each model separately); on Llama-3.1-8B it is above the 30K-token full context (44.93 vs. 44.70). Most of the gain on the new tasks comes from passage retrieval (95.6 vs. 56.6 accuracy on Qwen2.5-14B, 59.5 vs. 12.5 on Llama-3.1-8B), where duplicated padding pushes candidate paragraphs out of the context, so that the correct one is often missing. On the summarisation and few-shot tasks, which need some relevant material rather than one specific passage, DEDUPPACK and chunk packing are on par (+0.25, n.s.): removing duplicates does not hurt tasks in which it has little to recover.

6.5 A second benchmark: book-length contexts

To test whether the findings transfer beyond LongBench, we run the frozen configuration on InfiniteBench's English novel QA and multiple-choice tasks (Zhang et al., 2024), whose contexts are about ten times longer (Table 6). Because relevant passages are scattered over a whole novel, chunk packing duplicates only 8–11% of its context here (vs. 23–29% on LongBench), so the diagnosis predicts a smaller gain. No parameter was changed. The frozen DEDUPPACK is the best method in three of the four model–budget settings and outperforms the pair packer by 4.54 F1 [3.02, 6.02] pooled ($p < 0.001$). Its advantage over dense chunk packing points in the same direction as on LongBench (+1.21 pooled, between +0.84 and +1.56 per setting) but, with 580 questions, is not significant. With only 1,024 tokens, DEDUPPACK matches or exceeds the models reading 30K tokens of the book

Packer (BGE ranking, 1,024 tokens)	Qwen-14B	Llama-8B	Step Δ	Dup.
Chunk packing	42.34	39.03		22.8
+ merge overlapping ranges	44.42	41.17	**+2.11*****	1.2
+ document-order emission	44.75	41.05	+0.11	1.2
+ sentence-aligned padding (= DEDUPPACK)	**45.18**	**41.67**	+0.52	1.3
DEDUPPACK without backfilling	44.84	41.24	−0.38	1.3
DEDUPPACK without merging	43.32	40.10	**−1.71*****	19.7

Table 7: Build-up ablation with a fixed BGE ranking. Step Δ: paired F1 difference to the row above (bottom rows: to DEDUPPACK), pooled over the two models (***: $p < 0.001$; significant on each model separately). Dup.: duplicated share of the packed context (%, Qwen tokenizer).

Method	Trained	Extra model	s / inst.	Device	GPU mem.
BGE-small encoding (shared)	no	33M	0.15	GPU	0.3 GB
+ chunk packing	no	–	+0.27	CPU	–
+ DEDUPPACK	no	–	+0.40	CPU	–
+ reranker (top 100)	no	568M	+0.16	GPU	1.3 GB
+ Provence pruning	yes	DeBERTa-v3	+0.76	GPU	3.4 GB
LLMLingua-2	yes	560M	0.87	GPU	3.3 GB
LongLLMLingua	no	8B	21.05	GPU	18.4 GB

Table 8: Cost of building a 1,024-token context (100 random LongBench test instances, one at a time, one A100 to itself; DEDUPPACK and chunk packing run on one CPU core). The downstream model's prefill cost is identical for all methods because the budget is identical; it is about 12× smaller than for the full context.

(38.03 vs. 36.44 for Qwen2.5-14B, 38.36 vs. 38.16 for Llama-3.1-8B).

6.6 How close to the full context?

At 2,048 tokens, DEDUPPACK is statistically indistinguishable from reading the full context on all three models: the paired differences are $+0.56$ $[-1.46, 2.56]$ for Qwen2.5-7B, -0.35 $[-2.39, 1.65]$ for Llama-3.1-8B and -0.10 $[-2.04, 1.82]$ for Qwen2.5-14B. The packed context is about one sixth of the mean prompt (2,048 vs. 12.5K tokens). On the multi-hop tasks it exceeds the full context on all three models (49.2 vs. 44.4, 45.8 vs. 43.7 and 56.0 vs. 52.6 MH F1), consistent with the observation that long inputs dilute the evidence that matters (Liu et al., 2024a). At 1,024 tokens, DEDUPPACK recovers 93–95% of full-context F1.

6.7 Evidence-level vs. token-level compression

Under the same exact 1,024-token budget, LLMLingua-2 reaches 24.6–28.4 F1, and DEDUPPACK exceeds it by 15.27 F1 [13.90, 16.64] pooled over models. At roughly 12× compression a token-level compressor must remove most of every sentence, including the names, numbers and function words that extractive QA needs to copy, whereas DEDUPPACK spends the budget on a few complete passages. Appendix E shows the same ordering for Selective Context and LongLLMLingua.

6.8 Deduplication is the active ingredient

To find out which operation produces the gain, we start from standard chunk packing and add one operation at a time, keeping the BGE ranking fixed so that only the packing changes (Table 7; our re-implementation of chunk packing reproduces the baseline of Table 4 within 0.04 F1). Merging overlapping ranges alone raises F1 by 2.11 [1.16, 3.09] ($p < 0.001$), on each model separately, and removes almost all duplication (22.8%→1.2%). It accounts for 77% of the full gain of 2.74 F1. Document-order emission and sentence-aligned padding add 0.11 and 0.52 F1, and backfilling 0.38; none of these steps is significant on its own. Conversely, removing merging from the full packer costs 1.71 F1 [0.76, 2.66] ($p < 0.001$) and brings duplication back to 20%. Merging alone is also significant at the other budgets (+1.47 at 512 and +1.64 at 2,048 tokens, both $p \leq 0.002$). We keep the secondary operations because they never hurt significantly and make the packed context more readable, but the evidence is clear that the accuracy gain comes from not paying for the same text twice.

More duplication, larger gain. If deduplication drives the gain, instances whose chunk-packed context is more redundant should benefit more. We split the test instances into terciles by the duplication rate of their chunk-packed context. At 512 tokens, the F1 gain of DEDUPPACK over chunk packing (same BGE ranking) grows from -0.2 and +1.7 in the least redundant tercile to +6.8 and +7.1 in the most redundant one (Qwen2.5-7B and Llama-3.1-8B), and the gain in answer coverage grows monotonically with duplication at 512 and 1,024 tokens on both models (e.g. +1.0, +3.1 and +7.8 points for Qwen2.5-7B at 512). The same mechanism explains the smaller gains on InfiniteBench (Section 6.5): there, relevant passages are scattered over a whole novel, and chunk packing duplicates only 8–11% of its context.

6.9 Cost

Table 8 measures the cost of each context-construction method under controlled conditions. DEDUPPACK adds 0.40 s of single-core CPU time to the shared BGE encoding pass (0.15 s on the GPU), 0.13 s more than chunk packing, and needs neither training nor GPU memory beyond the 33M-

parameter encoder. Provence and LLMLingua-2 require a trained pruning or compression model and take about 1.6 times as long end to end, and LongLLMLingua, which scores a long context with an 8B language model, takes 21 s per instance and 18 GB of GPU memory—about 40 times the cost of DEDUPPACK for 2.5 F1 less. Every DEDUPPACK context respects the exact budget (maximum 1,024 tokens) and fills it almost completely (1,014 tokens on average vs. 987 for chunk packing).

7 Discussion

Why does packing matter so much? Under an exact budget, every duplicated token is a token not spent on evidence. Standard packing pads each top-ranked chunk independently, but top-ranked chunks are not independent: they cluster in the relevant passages, so their padding overlaps and a quarter of the budget is spent twice. Merging the padded ranges removes this waste and is worth about 2 F1 on its own, as much as replacing BGE-small with BGE-large or adding a cross-encoder reranker. Simpler designs that avoid overlap by construction, such as non-overlapping sentence chunks, recover part of the gain, consistent with this diagnosis; DEDUPPACK achieves it behind any existing chunking and ranker, without re-indexing.

Implications for compression research. Most work on context compression compares rankers or learned scorers with a fixed, simple packer. Our results suggest that such comparisons should hold the packer fixed *and* strong, because a better packer gives every ranker a similar boost. DEDUPPACK needs only the downstream tokenizer and adds no model calls, so it can be adopted as the default packer behind any retriever or learned scorer.

Deployment. DEDUPPACK is training-free, runs before the model is called and outputs verbatim source text in document order, so the compressed context can be shown to users and traced back to its sources.

Practical implications. DEDUPPACK is a drop-in replacement for the packing step: it needs no training and no model beyond the ranker, and adds about 0.4 s of CPU time per request (Section 6.9). Its savings translate directly into input tokens. At half the budget, DEDUPPACK nearly matches chunk packing: with 1,024 tokens it scores 41.98 F1 pooled over the three models, against 42.37 for chunk packing with 2,048 tokens, and on Llama-3.1-8B it is higher (40.98 vs. 40.85). Interpolating between budgets, chunk packing needs roughly 1.6–2× as many tokens to reach the accuracy of a 1,024-token DEDUPPACK context. Against the full context, a 2,048-token DEDUPPACK context keeps 99–101% of F1 with one sixth of the input, which reduces prefill cost and the per-request KV cache by about the same factor. The benefit is largest in pipelines that expand retrieved hits with neighbouring text, such as sentence-window or parent-document retrieval, where we measure 9–30% duplication; with plain, non-overlapping chunks drawn from many different documents duplication is low and there is little to remove.

8 Conclusion

We studied long-context question answering under a hard budget measured in the downstream model's own tokens and showed that the step that turns ranked spans into a context is a major, understudied lever. Standard packing spends 15–30% of the budget on duplicated text; DEDUPPACK, whose core is merging overlapping evidence so that no text is paid for twice, improves over dense chunk packing in all nine model–budget settings we test, by 1.9–3.7 F1 pooled over models, transfers across lexical, dense and hybrid rankers, and with one sixth of the tokens retains 99–101% of full-context F1 on three LLMs. We hope this encourages treating packing as a first-class component of context compression.

Limitations

DEDUPPACK shortens the input before prefill and is not a KV-cache method. Our experiments cover English extractive QA on LongBench with three instruction models of 7–14B parameters and budgets of 512–2,048 tokens; other languages, generation tasks such as summarisation, and much larger models may behave differently. The development split covers two multi-hop tasks, and the method was frozen before testing; a broader development set might lead to a different choice of ranker, although our results suggest the ranker matters less than the packer. Apart from merging, the individual packing operations have small effects that are not significant on their own. The dense and hybrid rankers require a small encoder at inference time.

Ethics Statement

Context compression can remove information a user would consider important, which can lead to incomplete or incorrect answers; in high-stakes settings, compressed contexts should be shown to the user or checked against the source. DEDUPPACK keeps all admitted text verbatim, which makes such checks straightforward. All datasets are public research benchmarks used under their licences.

References

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2023. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*.

Jaime Carbonell and Jade Goldstein. 1998. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 335–336.

Nadezhda Chirkova, Thibault Formal, Vassilina Nikoulina, and Stéphane Clinchant. 2025. Provence: Efficient and robust context pruning for retrieval-augmented generation. In *The Thirteenth International Conference on Learning Representations*.

Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. 2021. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4599–4610.

Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman and Hall/CRC.

Matoš Eibich, Shivay Nagpal, and Alexander Fred-Ojala. 2024. ARAGOG: Advanced RAG output grading. arXiv:2404.01037.

Aaron Grattafiori et al. 2024. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625.

Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pages 874–880.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. Llmlingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736*.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. *arXiv preprint arXiv:2310.06839*.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781.

Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. 2018. The NarrativeQA reading comprehension challenge. *Transactions of the Association for Computational Linguistics*, 6:317–328.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.

Yucheng Li, Bo Dong, Chenghua Lin, and Frank Guerin. 2023. Selective context: Efficient context selection for large language models. *arXiv preprint arXiv:2304.12102*.

Yucheng Li, Ying Huang, Bowen Yang, Bharat Venkitesh, Andrea Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*.

Barys Liskavets, Maxim Ushakov, Shuvendu Roy, Mark Klibanov, Ali Etemad, and Shane K. Luke. 2025. Prompt compression with context-aware sentence encoding for fast and improved LLM inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 24595–24604.

Jerry Liu. 2022. LlamaIndex. https://github.com/run-llama/llama_index.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024a. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024b. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*.

Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. 2024. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 963–981.

Qwen Team. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Sietse Schelpe. 2026. Byte-exact deduplication in retrieval-augmented generation: A three-regime empirical analysis across public benchmarks. arXiv:2605.09611.

Nataraj Agaram Sundar and Tejas Morabia. 2026. Self-conditioned positional HNSW for overlap-aware retrieval in chunked-document RAG systems: Method and industrial evidence-quality audit. arXiv:2606.01542.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multi-hop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 10014–10037.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023a. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*.

Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023b. C-pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*.

Wenhan Xiong, Xiang Lorraine Li, Srinivasan Iyer, Jingfei Du, Patrick Lewis, William Yang Wang, Yashar Mehdad, Wen-tau Yih, Sebastian Riedel, Douwe Kiela, and Mohit Bansal. 2021. Answering complex open-domain questions with multi-hop dense retrieval. In *Proceedings of the International Conference on Learning Representations*.

Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2024. Recomp: Improving retrieval-augmented lms with compression and selective augmentation. In *Proceedings of the International Conference on Learning Representations*.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380.

Chanwoong Yoon, Taewhoo Lee, Hyeon Hwang, Minbyul Jeong, and Jaewoo Kang. 2024. CompAct: Compressing retrieved documents actively for question answering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 21424–21439.

Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Khai Hao, Xu Han, Zhen Leng Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. 2024. ∞Bench: Extending long context evaluation beyond 100K tokens. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Re, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *arXiv preprint arXiv:2306.14048*.

A Implementation Details

Algorithm 1 DEDUPPACK

Require: question q, context C, budget B, tokenizer τ, ranker f
split C into 256-token pages and 64-token spans (stride 32)
$P \leftarrow$ top 60 spans by $f(q, s)$
$\mathcal{S} \leftarrow \emptyset$
for $s \in P$ in order **do**
    $R \leftarrow$ sentence-repaired ranges of $\mathcal{S} \cup \{s\}$ ($\leq$32 tokens per side, within page)
    merge overlapping ranges of R; render in document order after the prefix
    if $|\text{render}|_\tau \leq B$ **then** $\mathcal{S} \leftarrow \mathcal{S} \cup \{s\}$
    end if ▷ otherwise skip s and continue (backfill)
end for
return render of $\mathcal{S}$, clipped to B tokens if needed

Inference used vLLM 0.7.3 with bfloat16 weights on NVIDIA A100-40GB GPUs (tensor parallelism of two for the 14B full-context run), with a 4,096-token model window for compressed contexts and 32,768 tokens for the full-context reference. Code, packed contexts and per-instance predictions for every number in this paper will be released.

B Development Results

Table 10 reports macro F1 on the development split (HotpotQA-E and 2WikiMultihopQA-E without

Item	Value
Pages / spans	256 regex tokens / 64, stride 32
Lexical ranker	$0.5\,z(\mathrm{BM25}) + 0.5\,z(\mathrm{TFIDF})$; k_1=1.2, b=0.75; sublinear TF, smoothed IDF
Dense ranker	`bge-small-en-v1.5`, CLS pooling, ℓ_2-normalised, query instruction
Hybrid ranker	$z(\text{lexical}) + z(\text{dense})$
Priority list	top 60 spans
Sentence terminators	`. ? !`
Repair radius	32 regex tokens per side, within page
Admission	rendered context under τ, backfill
Emission	document order, blank-line separated

Table 9: Implementation constants (all frozen before testing).

test questions, 584 instances, 1,024 tokens, 4-bit Qwen2.5-7B-Instruct). These results determined the frozen configuration; the hybrid ranker with sentence repair was the best development configuration, +5.26 F1 [2.34, 8.23] above BGE chunk packing.

Ranker (all with merge + backfill)	Fixed $\pm$32	Sentence
Lexical	45.37	47.90
BGE	48.88	50.12
Hybrid	48.17	**51.19**
BGE chunks	45.93	
Pair packing	45.03	

Table 10: Development macro F1 (1,024 tokens).

C Per-Task Results

Table 11 lists per-task F1 at 1,024 tokens. Multi-hop (MH) F1 of DEDUPPACK (hybrid) at 512 / 2,048 tokens is 39.47 / 49.20 (Qwen2.5-7B), 37.02 / 45.78 (Llama-3.1-8B) and 44.18 / 56.01 (Qwen2.5-14B), against 34.84 / 46.37, 33.52 / 44.14 and 40.02 / 52.57 for BGE chunks.

D Per-Model Paired Tests

E Additional Baselines

In an earlier run with the same candidate spans, prompt wrapper, scorer and exact 1,024-token budget, but with Qwen2.5-7B-Instruct and Meta-Llama-3-8B-Instruct in an earlier software environment, the pair packer of Table 2 outperformed head and middle truncation by 13.1 and 13.6 F1, Selective Context by 18.2, LLMLingua-2 by 11.3, LongLLMLingua by 6.6, and BM25 and TF-IDF chunk packing by 2.1 and 1.9 F1 (two-model averages, all $p < 0.001$). Since DEDUPPACK outperforms the pair packer in every setting of Table 2, these baselines are dominated by DEDUPPACK as well; we report them here rather than in the main table because they were run in a different environment.

Model	Method	Qasper	MF-en	HotpotQA	2Wiki	MuSiQue	NarrQA	MH	Avg.
Qwen2.5-7B	Full context	44.86	51.41	58.07	45.61	29.36	28.94	44.35	43.04
	LLMLingua-2	33.23	31.00	32.80	29.19	13.19	8.16	25.06	24.60
	Pair packing	40.40	50.08	54.05	46.25	26.83	17.05	42.38	39.11
	BGE chunks	38.90	47.90	51.61	45.88	30.72	18.08	42.74	38.85
	DEDUPPACK (BGE)	42.03	46.40	57.00	52.41	29.89	16.82	46.43	40.76
	DEDUPPACK (hybrid)	38.73	49.31	57.10	48.32	33.39	16.97	46.27	40.64
Llama-3.1-8B	Full context	45.28	54.80	55.79	44.14	31.26	27.52	43.73	43.13
	LLMLingua-2	34.31	33.35	37.35	31.53	20.40	13.61	29.76	28.43
	Pair packing	38.33	50.09	49.47	45.11	24.25	19.46	39.61	37.79
	BGE chunks	40.65	48.49	47.33	48.11	28.50	20.86	41.31	38.99
	DEDUPPACK (BGE)	41.79	49.78	56.18	49.58	31.84	20.85	45.87	41.67
	DEDUPPACK (hybrid)	41.28	52.99	51.86	49.42	30.37	19.95	43.89	40.98
Qwen2.5-14B	Full context	45.46	53.86	62.00	58.52	37.21	28.50	52.58	47.59
	LLMLingua-2	30.93	32.71	38.34	34.15	18.34	8.12	30.27	27.10
	Pair packing	39.37	49.88	56.88	52.25	31.82	19.92	46.98	41.69
	BGE chunks	39.75	49.05	55.42	53.34	35.28	21.19	48.02	42.34
	DEDUPPACK (BGE)	41.29	50.37	60.59	60.29	37.33	21.18	52.74	45.18
	DEDUPPACK (hybrid)	40.70	53.03	58.78	55.52	36.65	21.24	50.32	44.32

Table 11: Per-task F1 at 1,024 tokens (full context: up to 30K tokens).

Model	Comparison	512	1,024	2,048
Qwen2.5-7B	DEDUPPACK − BGE chunks	+3.69 [1.69, 5.68]	+1.79 [−0.08, 3.65]	+2.62 [0.82, 4.49]
	DEDUPPACK − Pair packing	+2.86 [1.09, 4.59]	+1.53 [−0.14, 3.17]	+2.92 [1.26, 4.58]
	DEDUPPACK-BGE − BGE chunks	+2.59 [0.91, 4.29]	+1.91 [0.21, 3.64]	+2.17 [0.43, 3.96]
Llama-3.1-8B	DEDUPPACK − BGE chunks	+3.17 [1.14, 5.17]	+1.99 [0.20, 3.84]	+1.93 [0.08, 3.80]
	DEDUPPACK − Pair packing	+2.39 [0.69, 4.07]	+3.19 [1.48, 4.89]	+3.18 [1.53, 4.80]
	DEDUPPACK-BGE − BGE chunks	+2.89 [1.24, 4.56]	+2.68 [1.06, 4.38]	+2.73 [1.03, 4.44]
Qwen2.5-14B	DEDUPPACK − BGE chunks	+4.17 [2.27, 6.12]	+1.98 [0.20, 3.85]	+2.21 [0.45, 3.99]
	DEDUPPACK − Pair packing	+3.71 [2.15, 5.26]	+2.63 [1.17, 4.12]	+3.27 [1.65, 4.90]
	DEDUPPACK-BGE − BGE chunks	+2.95 [1.38, 4.53]	+2.84 [1.25, 4.49]	+2.81 [1.12, 4.59]

Table 12: Per-model paired bootstrap differences (F1, 95% intervals). DEDUPPACK denotes the frozen hybrid configuration.